

Week 2 – Day 1 Assignments

1. Try prompting using TL;DR and ELI-15 and observe the difference in output.

LLM/Model used: Claude – Sonnet 4.6

Regular Prompt: What is playwright? How can I use it for automation testing?

LLM response:

What is Playwright?

Playwright is an open-source browser automation framework developed by Microsoft. It allows you to write scripts that control web browsers (Chromium, Firefox, and WebKit) programmatically — making it ideal for **end-to-end (E2E) testing**, **web scraping**, and **UI automation**.

Key Features

- **Cross-browser** — works with Chromium, Firefox, and Safari (WebKit) from a single API
- **Cross-language** — supports JavaScript/TypeScript, Python, Java, and .NET
- **Auto-waits** — automatically waits for elements to be ready before acting (no manual sleep() calls)
- **Headless & headed modes** — run with or without a visible browser window
- **Network interception** — mock APIs, intercept requests, simulate offline mode
- **Parallel execution** — run tests concurrently across browsers and devices
- **Trace viewer** — record and replay test runs with screenshots, videos, and logs

Getting Started

1. Install Playwright

```
bash
```

```
npm init -y
```

```
npm install -D @playwright/test
```

```
npx playwright install # installs browser binaries
```

2. Write Your First Test

```
typescript
```

```
// tests/example.spec.ts
```

```
import { test, expect } from '@playwright/test';
```

```
test('homepage has correct title', async ({ page }) => {  
  await page.goto('https://example.com');  
  await expect(page).toHaveTitle(/Example Domain/);  
});
```

```
test('button click works', async ({ page }) => {  
  await page.goto('https://example.com');  
  await page.click('text=More information');  
  await expect(page).toHaveURL(/iana.org/);  
});
```

3. Run Tests

bash

```
npx playwright test          # run all tests  
npx playwright test --headed  # run with visible browser  
npx playwright test --browser=firefox # specific browser  
npx playwright test --ui      # open interactive UI mode
```

Core Concepts

Locators (finding elements)

typescript

```
page.getByRole('button', { name: 'Submit' }) // semantic (preferred)
```

```
page.getByText('Login')
```

```
page.getByPlaceholder('Enter email')
```

```
page.getByTestId('submit-btn')
```

```
page.locator('.css-class') // CSS selector
```

```
page.locator('//xpath') // XPath
```

Assertions

typescript

```
await expect(page).toHaveURL('https://...');
```

```
await expect(page).toHaveTitle('...');
```

```
await expect(locator).toBeVisible();
```

```
await expect(locator).toHaveText('Hello');

await expect(locator).toBeEnabled();

await expect(locator).toHaveValue('input value');
```

Actions

```
typescript

await page.goto('https://example.com');

await page.click('button');

await page.fill('input[name="email"]', 'user@example.com');

await page.selectOption('select', 'option-value');

await page.check('input[type="checkbox"]');

await page.keyboard.press('Enter');

await page.screenshot({ path: 'screenshot.png' });
```

API Mocking

```
typescript

await page.route('**/api/users', route => {
  route.fulfill({ json: [{ id: 1, name: 'Alice' }] });
});
```

Configuration (playwright.config.ts)

```
typescript

import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30_000,
  retries: 2,
  workers: 4,           // parallel workers
  reporter: 'html',
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
  ],
});
```

```
{ name: 'mobile', use: { ...devices['iPhone 13'] } },  
],  
use: {  
  baseUrl: 'http://localhost:3000',  
  screenshot: 'only-on-failure',  
  video: 'retain-on-failure',  
  trace: 'on-first-retry',  
},  
});
```

Useful CLI Commands

Command	Description
<code>npx playwright test</code>	Run all tests
<code>npx playwright test --ui</code>	Interactive UI mode
<code>npx playwright codegen</code>	Record tests by clicking in browser
<code>npx playwright show-report</code>	Open HTML test report
<code>npx playwright show-trace trace.zip</code>	Inspect a trace

Tips for Good Automation Tests

1. **Use `getByRole` / `getByTestId`** over CSS selectors — more resilient to UI changes
2. **Avoid hardcoded waits** (`sleep`) — rely on Playwright's auto-wait
3. **Use Page Object Model (POM)** to organize large test suites
4. **Run in CI** with `npx playwright test --reporter=github` for GitHub Actions
5. **Use `codegen`** to record interactions and auto-generate test code

Playwright is widely regarded as the most powerful and developer-friendly E2E testing tool available today — a strong upgrade over older tools like Selenium or Cypress.

.....

LLM/Model used: Claude – Sonnet 4.6

TL;DR Prompt: What is playwright? How can I use it for automation testing?

LLM response:

Playwright is Microsoft's open-source browser automation framework for end-to-end testing across Chromium, Firefox, and WebKit.

Get started in 3 steps:

1. **Install** — `npm install -D @playwright/test && npx playwright install`
2. **Write a test:**

`typescript`

```
import { test, expect } from '@playwright/test';
```

```
test('page loads', async ({ page }) => {  
  await page.goto('https://example.com');  
  await expect(page).toHaveTitle(/Example/);  
});
```

3. **Run** — `npx playwright test`

Why use it? Auto-waits for elements, parallel execution, built-in network mocking, and a codegen tool that records your clicks and generates test code automatically.

Understanding:

For the Regular prompt, the LLM has provided very long detailed answer. It gave the installation steps and also additionally it gave the explanation on Key Features, Core Concepts like Locators, Assertions, Actions, API Mocking etc.,

But while using TL;DR in the prompt the LLM has provided very short response that comprises of the installation, basic sample test to run and check the installation and run command for the test. No other details have been given while using TL;DR.

.....

LLM/Model used: Claude – Sonnet 4.6

ELI15 Prompt: What is playwright? How can I use it for automation testing?

LLM response:

Imagine you could write a script that controls a web browser like a robot — clicking buttons, filling forms, and checking if pages look right — **that's Playwright**.

The analogy

Think of it like a **cheat code for testing websites**. Instead of manually opening Chrome and clicking around every time you change your code, you write a script once and Playwright does all the clicking for you, automatically, every time.

A real example

typescript

```
test('login works', async ({ page }) => {  
  await page.goto('https://myapp.com/login');  
  await page.fill('#email', 'user@example.com'); // types in the box  
  await page.fill('#password', 'secret123');  
  await page.click('button[type="submit"]'); // clicks login  
  await expect(page).toHaveURL('/dashboard'); // checks it worked  
});
```

That's literally a robot logging into a website and verifying it worked. No human needed.

Why is it cool?

- 🤖 **Automates the boring stuff** — no manual clicking to test your app
 - 🌐 **Works on Chrome, Firefox, and Safari** from the same script
 - ⚡ **Smart waiting** — it knows when a button is ready to click, no guessing
 - 📹 **Records your clicks** and writes the test code for you (npx playwright codegen)
-

Getting started

bash

```
npm install -D @playwright/test
```

```
npx playwright install    # downloads the browsers
```

```
npx playwright codegen    # open browser, click around, get free test code
```

```
npx playwright test      # run your tests
```

Basically — you click around your website once, Playwright watches and writes the test, then repeats it forever automatically. Pretty neat.

.....

Understanding:

Using ELI 15 in the prompt gives the response that is easy for a Novice user who has completely no idea of Playwright. Also, the content LLM has shared is very limited and does not talk about the configuration or setup. But it still highlights the key features which might attract the user to deep dive into individual other topics.

.....

2. Use structured reasoning / step-by-step prompting to generate:

a) Automation Framework Architecture

b) Test Strategy Document

a) **Automation Framework Architecture:**

LLM/Model : Claude Sonnet 4.6

Prompt :

Context: Act as an Automation Architect having strong expertise in Embedded Testing ,
Manufacturing domain expertise and automation framework design experience in Selenium Java -
Cucumber experience

Instructions:

[MANDATORY] Ask me 5 multiple choice questions one by one

[MANDATORY] Once after answering the first question, proceed with next

[MANDATORY] Generate framework structure First, after completing all the questions answered

[STRICT] Proceed step by step for framework implementation

[STRICT] Up on my approval proceed with the next step

[STRICT] Follow best practices of Selenium Java and implement good coding standards

[STRICT] Use Page Object Model as a Design Pattern

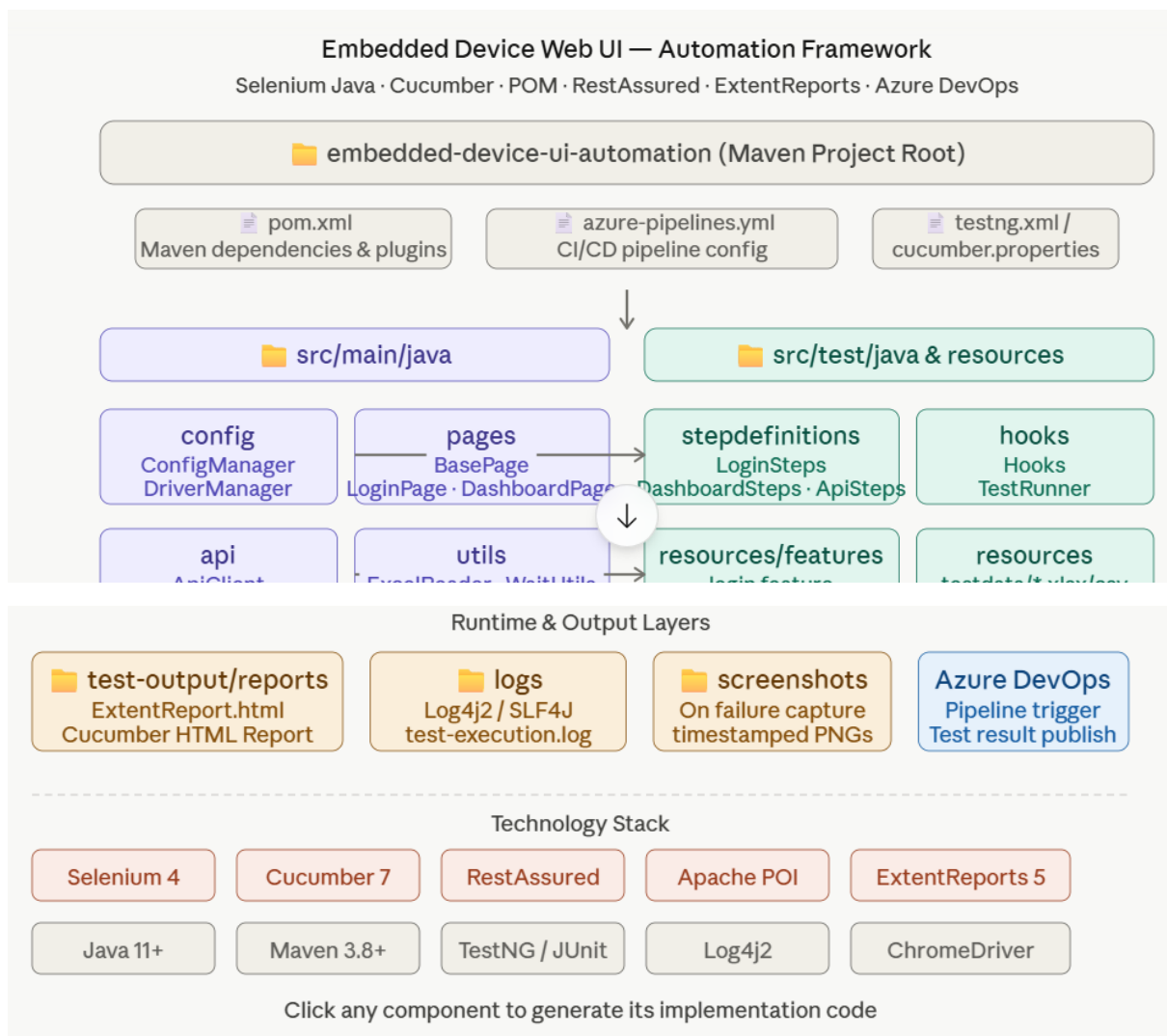
LLM Response :

All 5 questions answered! Let me compile your selections and generate the complete framework structure.

Requirements Summary

#	Area	Choice
1	Application Type	Embedded Device Web UI (ECU/MCU)
2	Test Scope	UI + REST API Validation
3	Reporting & CI/CD	Extent Reports + Azure DevOps
4	Test Data & Config	Excel/CSV + Single Environment Config
5	Browser & Execution	Chrome Only + Local Execution

Framework Structure



Complete Package Structure (Text Reference)

embedded-device-ui-automation/

```
|
|
| └─ pom.xml
|
| └─ azure-pipelines.yml
|
| └─ cucumber.properties
|
| └─ src/main/java/com/embedded/automation/
|   | └─ config/
|   |   | └─ ConfigManager.java      ← reads config.properties
|   |   |   └─ DriverManager.java    ← Chrome WebDriver singleton
|   |   └─ pages/
|   |       | └─ BasePage.java        ← POM base with common actions
|   |       |   └─ LoginPage.java
|   |       └─ DashboardPage.java
|   └─ api/
|       | └─ ApiClient.java          ← RestAssured base setup
|       |   └─ EcuApiService.java    ← ECU-specific REST endpoints
|       └─ utils/
|           | └─ ExcelReader.java     ← Apache POI for CSV/Excel
|           |   └─ WaitUtils.java     ← Explicit/Fluent waits
|           |   └─ ReportManager.java ← ExtentReports manager
|           └─ LogUtils.java          ← Log4j2 wrapper
|
| └─ src/test/java/com/embedded/automation/
|   | └─ stepdefinitions/
|   |   | └─ LoginSteps.java
|   |   |   └─ DashboardSteps.java
|   |   └─ ApiSteps.java
```

```

|   └─ hooks/
|   |   └─ Hooks.java           ← @Before @After Cucumber hooks
|   |   └─ TestRunner.java      ← Cucumber TestNG runner
|   └─ resources/
|       └─ features/
|           └─ login.feature
|           └─ dashboard.feature
|           └─ ecu_api.feature
|       └─ testdata/
|           └─ TestData.xlsx
|       └─ config.properties    ← ECU IP, browser, env config
|
└─ test-output/
    └─ reports/ExtentReport.html
    └─ screenshots/
    └─ logs/test-execution.log

```

Ready to Implement — Step by Step Plan

Step	Task	Artifact
Step 1	pom.xml — all dependencies	Maven config
Step 2	config.properties + ConfigManager.java	Config layer
Step 3	DriverManager.java	Chrome singleton
Step 4	BasePage.java + Page classes	POM layer
Step 5	ApiClient.java + EcuApiService.java	REST layer
Step 6	ExcelReader + WaitUtils + LogUtils	Utilities
Step 7	ReportManager.java + Hooks.java	Reporting
Step 8	Step Definitions + Feature files	Test layer
Step 9	TestRunner.java	Execution
Step 10	azure-pipelines.yml	CI/CD

Shall I proceed with Step 1 — pom.xml with all required dependencies? Type "proceed" or "yes" and I'll implement it with full best practices.

You said: yes

yes

12:28 AM

Claude responded: Step 1 — pom.

Step 1 — pom.xml

What's included & why

Selenium 4.18.1 — latest stable; native CDP support works well with Chromium-based embedded browsers on ECU.

WebDriverManager 5.7.0 — auto-resolves ChromeDriver binary, eliminating manual driver management across developer machines and CI agents.

Cucumber 7 + PicoContainer — PicoContainer enables dependency injection between step definition classes so they can share the same WebDriver instance cleanly without static variables.

RestAssured 5.4 — industry standard for REST API validation; pairs cleanly with Cucumber step definitions.

Apache POI 5.2.5 + OpenCSV — covers both .xlsx and .csv test data formats as per your requirement.

ExtentReports 5 + Cucumber7 Adapter — the adapter auto-hooks into Cucumber lifecycle events and generates rich HTML reports without manual wiring.

Log4j2 + SLF4J bridge — all third-party libraries that use SLF4J get routed to Log4j2 automatically.

Three Maven Profiles — smoke, regression (default), sanity — can be triggered from Azure DevOps pipeline with -P smoke for targeted runs.

CLI usage examples

bash

Run full regression suite (default)

mvn clean test

Run only smoke tests

mvn clean test -P smoke

Run with a custom Cucumber tag

mvn clean test -Dcucumber.filter.tags="@Login"

✅ **Step 1 complete.**

Step 2 is next: config.properties + ConfigManager.java — the configuration layer that manages ECU device IP, browser settings, timeouts, and API base URL.

Shall I proceed with **Step 2**?

Pom

XML

Download

You said: yes

yes

12:29 AM

Claude responded: Step 2 — Configuration Layer

Step 2 — Configuration Layer

config.properties + ConfigManager.java

Created 2 files, read a file

Created 2 files, read a file

What's implemented & why

config.properties covers five configuration zones:

Zone	Key Properties
ECU Device	ecu.host, ecu.base.url, ecu.api.base.url, firmware version
Browser	browser, browser.headless, window dimensions
Timeouts	implicit (0), explicit (20s), pageload (30s), polling (500ms)
REST API	auth type, credentials, content-type, connection/socket timeouts
Reporting	report paths, screenshot on failure, log directory

ConfigManager.java — key design decisions:

Double-checked locking singleton — thread-safe without synchronizing every call, important if parallel test runners are added later.

JVM system property precedence — any -Dkey=value passed from the Azure DevOps pipeline YAML overrides the file value. This means the same config.properties works locally and in CI without modification.

yaml

Azure DevOps pipeline example override

- script: mvn clean test -Decu.host=192.168.10.50 -Dbrowser.headless=true

Typed getters — every consumer calls `getExplicitTimeout()` not `getProperty("timeout.explicit")`, keeping magic strings out of test code entirely.

logConfiguration() — called once at suite startup in `Hooks.java` (Step 7), printing a clean summary to the log with sensitive values masked.

✅ **Step 2 complete.**

Step 3 is next: `DriverManager.java` — Thread-safe Chrome `WebDriver` singleton using `ThreadLocal`, with ECU-specific Chrome options, automatic driver management via `WebDriverManager`, and proper teardown.

Shall I proceed with **Step 3**?

Config

PROPERTIES

Download

Configmanager

Code · JAVA

Download

Download all

You said: yes

yes

12:32 AM

Claude responded: Step 3 — DriverManager.

Step 3 — DriverManager.java

Created a file, read a file

Created a file, read a file

What's implemented & why

ThreadLocal<WebDriver> — each test thread gets its own isolated driver instance. This costs nothing now with sequential execution, but enables parallel test runs later with zero refactoring.

ECU-specific Chrome flags — these are the critical ones for embedded device testing:

Chrome Flag	Why needed for ECU
--ignore-certificate-errors + setAcceptInsecureCerts(true)	ECU web servers almost always use self-signed SSL certs
--disable-web-security	ECU UI often makes cross-origin API calls to same device
--allow-running-insecure-content	Mixed HTTP/HTTPS is common on embedded web servers
--disable-images	ECU web servers are resource-constrained — speeds up page loads
--no-sandbox + --disable-dev-shm-usage	Required for Azure DevOps Linux agents and Docker

Implicit wait = 0 — a deliberate best practice. Mixing implicit and explicit waits causes unpredictable timeout behaviour. All waiting is done through WaitUtils (Step 6) using WebDriverWait.

Isolated user-data-dir per thread — prevents Chrome profile conflicts if two test processes run on the same CI agent simultaneously.

isDriverActive() — used in Hooks.java to guard against double-quit errors when a scenario fails before the driver is even opened.

Driver lifecycle in the framework

@Before (Hooks.java)	@After (Hooks.java)
▼	▼
DriverManager.initDriver()	DriverManager.quitDriver()
▼	▼
WebDriverManager.setup()	driver.quit()
ChromeDriver created	ThreadLocal.remove()
Timeouts applied	
Navigate → ECU base URL	

✅ **Step 3 complete.**

Step 4 is next: BasePage.java + LoginPage.java + DashboardPage.java — the full Page Object Model layer with reusable element interactions, smart waits, and ECU-aware page validation patterns.

Shall I proceed with **Step 4**?

Drivermanager

Code · JAVA

Download

You said: yes

yes

12:34 AM

Claude responded: Step 4 — Page Object Model Layer

Step 4 — Page Object Model Layer

BasePage.java → LoginPage.java → DashboardPage.java

Created 3 files, read a file

Created 3 files, read a file

What's implemented & why

BasePage.java — the foundation everything else builds on:

Method Group	ECU-specific reasoning
waitForPageLoad()	ECU web servers are resource-constrained — document.readyState check avoids race conditions
jsClick() fallback	Non-standard JS frameworks on embedded UIs often block standard Selenium clicks
jsSetValue() + dispatchEvent('change')	ECU pages use JS event listeners for form validation; sendKeys alone won't trigger them
getLocalStorageValue()	ECU web UIs store session tokens and device state in localStorage
waitForInvisibility()	Loading spinners appear during ECU firmware ops, sensor refreshes — must wait them out
waitUntil() fluent wait	Custom conditions for complex ECU state polling (e.g. wait until device status = ONLINE)
shortWait (5s)	Used for isDisplayed() / isPresent() — avoids 20s wait on optional elements

LoginPage.java — key patterns:

- loginAs() — one-call happy path used in @Before hooks for tests that need pre-auth
- attemptLoginWithInvalidCredentials() — stays on LoginPage, used for negative test scenarios

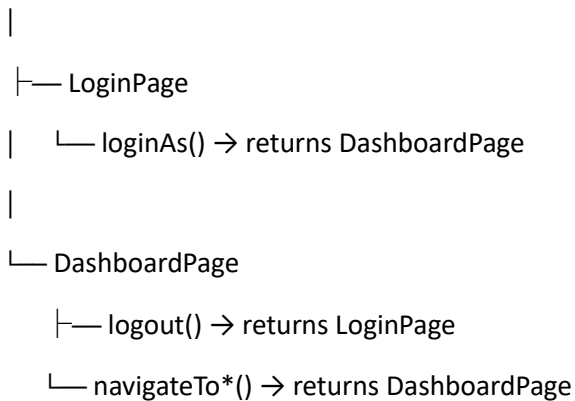
- Method chaining (enterUsername().enterPassword().clickLoginButton()) — readable step definitions

DashboardPage.java — ECU dashboard specifics:

- waitForDashboardLoad() in constructor — sensor data fetch causes a brief spinner; auto-handled before any assertion
- getSensorValue(label) — XPath-based lookup by sensor name, maps to any ECU I/O channel
- isDeviceOnline() — boolean convenience method for the most common dashboard assertion
- getWarningIndicatorCount() — checks for module-level warnings across network, I/O, CAN bus panels

POM layer relationship

BasePage (abstract)



✅ **Step 4 complete.**

Step 5 is next: ApiClient.java + EcuApiService.java — the REST API layer using RestAssured, with auth support, request/response logging, and ECU-specific endpoint methods for API validation alongside UI tests.

Shall I proceed with **Step 5**?

Basepage

Code · JAVA

Download

Loginpage

Code · JAVA

Download

Dashboardpage

Code · JAVA

Download

Download all

You said: yes

yes

12:37 AM

Downloadable files generated:

Pom.xml

Config.properties

Configmanager.java

Drivermanager.java

BasePage.java

LoginPage.java

Dashboardpage.java

b. Generate Test Strategy Document

LLM/Model Used : ChatGPT

Prompt:

Context: Act as an Test Manager for an Embedded Testing project which aims to design a test strategy that ensures its application for upgrading software in ECMs remotely is thoroughly tested in Web (UI and API), Mobile (Android, IOS)

[MANDATORY] Ask me 5 valid questions if you need clarity on writing test strategy

[STRICT] Once after answering first question, proceed with the next

[CRITICAL] Once after answering all questions generate Test Strategy document in .pdf format

[STRICT] Follow best practices to develop Test Plan and Strategy document [HIGH PRIORITY] Ensure both functional and non-functional(Accessibility, Performance) testing is covered

LLM Response:

Let's start with the first question.

Q1. What is the primary communication protocol used between the application and ECMs for remote software upgrades?

- A. HTTP/HTTPS (REST APIs)
- B. MQTT
- C. CAN / UDS (Unified Diagnostic Services)
- D. Proprietary protocol

Please answer (A/B/C/D), and I'll move to the next question.

A

Got it — using HTTP/HTTPS (REST APIs). Good, that simplifies some aspects of API and security testing.

Q2. What is the expected scale of ECM upgrade operations?

- A. Single ECM upgrade at a time per user
- B. Multiple ECMs (10–100) per batch
- C. Large-scale fleet upgrade (1000+ ECMs simultaneously)
- D. Mixed (both individual and large-scale fleet upgrades supported)

Please answer (A/B/C/D), and I'll proceed to the next question.

B

Understood — batch upgrades of 10–100 ECMs.

Q3. What level of rollback or recovery mechanism is supported if an upgrade fails?

- A. No rollback support
- B. Manual rollback by user
- C. Automatic rollback to previous stable version
- D. Partial recovery (retry from failure point)

Please answer (A/B/C/D), and I'll move to the next question.

B

Noted — rollback is manual.

Q4. What type of user roles are involved in the system?

- A. Single user role (Admin only)
- B. Multiple roles (Admin, Technician, Viewer)
- C. Role-based with fine-grained permissions (RBAC)
- D. External + internal users (e.g., dealers, OEMs, service engineers)

Please answer (A/B/C/D), and I'll proceed to the final question.

D

Got it — external and internal users involved.

Q5. What is the criticality level of the ECM upgrade operation?

- A. Low (failures are acceptable, no major impact)
- B. Medium (some impact, but recoverable easily)
- C. High (can impact vehicle/device functionality)
- D. Safety-critical (can impact safety or compliance)

Please answer (A/B/C/D), and I'll generate your **Test Strategy Document in PDF format** next.

C

Your Test Strategy document has been generated.

 [Download the PDF](#)

If you want, I can enhance this further to make it **interview-ready / enterprise-grade**, including:

- Detailed **test scenarios & traceability matrix**
- **API contract testing strategy**
- **CI/CD + Kubernetes integration plan**
- **AI-based test optimization (based on your interests)**

I find this strategy document does not have much details. Please give a detailed test strategy document with industry standards

You're right to expect more depth—what I gave earlier was too high-level for a Test Manager deliverable.

I've now created a **much more detailed, industry-aligned Test Strategy document** covering:

- Full **test lifecycle (levels, types, entry/exit)**
- Deep **functional + non-functional coverage**
- **Performance, security, accessibility**
- **Automation & CI/CD strategy (aligned to your background)**
- **Risk-based testing & mitigation**
- **Standards like ISO 26262, OWASP**
- **Batch upgrade validation (10–100 ECMs)**
- **Manual rollback considerations (critical gap area)**

👉 **Download the detailed PDF here:**

[Download Detailed ECM Test Strategy](#)

If you want to take this to **top 1% Test Manager level**, I can further enhance with:

- Traceability Matrix (RTM)
- Detailed **test scenarios per module**
- **API contract + schema validation strategy**
- **Chaos testing (network failure simulation for ECM upgrades)**
- Integration with **Kubernetes + AI-based test optimization** (aligned to your interests)

Test Strategy Document Embedded:

1. Objective

To define a comprehensive, risk-based test strategy for validating remote ECM software upgrade functionality across Web, Mobile (Android/iOS), and API layers ensuring reliability, performance, and safety.

2. Scope

In Scope: UI, API, mobile apps, backend services, batch upgrade (10–100 ECMs), authentication, authorization, upgrade orchestration. Out of Scope: Hardware-level ECU validation, manufacturing-level flashing.

3. System Overview

The system enables remote ECM firmware upgrades via REST APIs using HTTPS. Users include external dealers and internal engineers. Manual rollback is supported.

4. Test Levels

Unit Testing (Dev), Integration Testing, System Testing, End-to-End Testing, UAT, Regression Testing.

5. Functional Testing Strategy

• Validate login, role-based access • Upload firmware • Schedule/trigger upgrades • Monitor upgrade status • Batch execution (10–100 ECMs) • Failure handling & manual rollback • API contract validation • Negative scenarios (invalid firmware, network drop)

6. Non-Functional Testing Strategy

Performance: Validate system under batch load (100 ECMs), API latency < SLA. Scalability: Horizontal scaling validation. Security: OAuth2/JWT, penetration testing, OWASP Top 10. Accessibility: WCAG 2.1 compliance. Reliability: Retry mechanisms, failure recovery. Usability: UI responsiveness and user workflows.

7. Test Data Management

Use anonymized production-like data. Maintain firmware versions, device IDs, user roles. Ensure data isolation.

Just tell me 👍

Understanding:

In this exercise, I was able to relate the concept of COT (Chain of thoughts) and why its important. This gives a human in loop approach where LLM and a human can interact, which helps the LLM to clarify with the user before it generates rather than assuming and creating content on its own. Also any assumption or hallucination made by LLM will lead to loss of Tokens which indirectly impacts the cost factor.

